

JAVASCRIPT

1. What is JavaScript, and how does it differ from Java?

- JavaScript is a high-level, interpreted scripting language primarily used for web development. It runs in web browsers and allows developers to add interactivity and dynamic behavior to websites.
- Java, on the other hand, is a statically typed, compiled language used for various purposes, including web development. Despite their similar names, JavaScript and Java are entirely different languages with distinct syntax, use cases, and ecosystems.

2. Explain the difference between "null" and "undefined" in JavaScript?

- `null` is a deliberate assignment indicating the absence of a value or an empty object reference.
- `undefined` means that a variable has been declared but hasn't been assigned any value yet, or a function doesn't return anything explicitly.

3. How do you declare variables in JavaScript using "var," "let," and "const"? What are the differences between them?

- `var`: Function-scoped, can be redeclared, and has hoisting.
- `let`: Block-scoped, can be reassigned but not redeclared, and has no hoisting.
- `const`: Block-scoped, cannot be reassigned or redeclared, and has no hoisting.

4. What is the purpose of the "this" keyword in JavaScript, and how does its value change in different contexts?

- `this` refers to the current object or context. Its value changes depending on how and where it's used, such as in a function, method, or event handler.

5. What are callbacks in JavaScript? How do they work, and why are they important?

- Callbacks are functions passed as arguments to other functions. They're crucial for asynchronous operations, allowing you to execute code after a task completes, like handling HTTP requests or timers.

6. What is the purpose of the Document Object Model (DOM) in web development, and how do you manipulate it using JavaScript?

- The DOM represents the structure of a web page as a tree of objects, making it possible

to access and manipulate HTML and XML documents with JavaScript. You can change content, styles, and structure using DOM methods and properties.

7. What are the differences between `SetTimeout` and `setInterval`?

- `setTimeout` executes a function once after a specified delay.
- `setInterval` repeatedly executes a function at specified intervals until cleared.

8. What are the differences between `confirm`, `prompt`, and `alert`?

- `confirm` displays a dialog box with "OK" and "Cancel" buttons.
- `prompt` displays a dialog box for user input.
- `alert` displays a dialog box with a message and an "OK" button.

9. Describe the differences between "function expressions" and "function declarations."

- Function Declarations are hoisted and can be called before their declaration in the code.
- Function Expressions are not hoisted and can only be called after they're defined.

10. What is a synchronous and asynchronous function?

- Synchronous functions execute in sequence, blocking the program until they complete.
- Asynchronous functions allow the program to continue while they perform tasks in the background, notifying the program upon completion.

11. What are arrow functions in JavaScript, and how do they differ from regular functions?

- Arrow functions are concise function expressions introduced in ES6.
- They have a shorter syntax, automatically bind `this`, and cannot be used as constructors.

12. What is the difference between `=="` and `===` in JavaScript?

- `==` compares values after type coercion, attempting to make them of the same type.
- `===` compares values strictly, ensuring both value and type match.

13. What is an event in JavaScript, and how do you handle events in the browser using JavaScript?

- Events are interactions or occurrences in the browser, like clicks or key presses.
- You can handle events by attaching event listeners to DOM elements to execute code in response to events.

14. What is a loop, and how do you use "for" and "while" loops in JavaScript to iterate over data?

- Loops are control structures that execute a block of code repeatedly.
- `for` and `while` loops are used for iteration, with different syntax and use cases.

15. What is an object in JavaScript, and how do you create and work with objects and their properties?

- Objects are data structures containing key-value pairs.
- You can create objects using literals or constructors, access properties with dot notation or bracket notation, and modify them dynamically.

16. What is a data type in JavaScript? Can you name some primitive data types in JavaScript?

- A data type defines the kind of data a variable can hold.
- Primitive data types in JavaScript include `number`, `string`, `boolean`, `null`, `undefined`, and `symbol`.

17. What are arrays in JavaScript, and how do you create and manipulate them?

- Arrays are ordered collections of values accessed by index.
- You create arrays with square brackets and manipulate them using methods like `push`, `pop`, `splice`, and more.

18. Explain what JSON is, and how do you parse JSON data in JavaScript?

- JSON (JavaScript Object Notation) is a lightweight data interchange format.
- You can parse JSON data in JavaScript using `JSON.parse()` to convert a JSON string into a JavaScript object.

19. What is the ternary (conditional) operator in JavaScript, and how is it used to create concise conditional statements?

- The ternary operator (``?``) is a concise way to write conditional (if-else) statements.
- It evaluates an expression and returns one of two values based on the result.

20. What is the Document Object Model (DOM), and how do you interact with it using JavaScript?

- The DOM represents the structure of a web page as a tree of objects.
- You interact with it by accessing and modifying elements, attributes, and content using JavaScript.

21. Advantages and disadvantages of JavaScript:

Advantages:

- Versatile and widely used for web development.
- Supports both front-end and back-end development.
- Large and active developer community.
- Cross-platform compatibility.
- Supports asynchronous programming for responsive web applications.

Disadvantages:

- Limited security due to client-side execution.
- Browser compatibility issues.
- Lack of robust built-in libraries for certain tasks.
- Can lead to performance issues if not optimized.
- Requires careful error handling for stability.